BDI System Integration Points

Date: October 4, 2025
Repository: Mecca-Research/The-Binary-Decomposition-Interface
Purpose: Detailed documentation of all system integration points

---

Overview

This document provides a comprehensive map of all integration points in the BDI system, including interfaces, data formats, protocols, and implementation status.

---

1. VM ↔ Garbage Collector Integration

1.1 Interface Overview

Purpose: Enable automatic memory management in the VM through garbage collection.

Status: ❌ Not Implemented (Planned for PR #104)

1.2 Integration Points

A. Memory Allocation

Interface: `vm_allocate_object()`

```
// VM calls GC for object allocation
GCObject* vm_allocate_object(VM* vm, size_t size, uint32_t type_id) {
    GCObject* obj = mark_sweep_allocate(vm->gc, size, type_id);
    if (obj == NULL) {
        // Trigger GC and retry
        vm_trigger_gc(vm);
        obj = mark_sweep_allocate(vm->gc, size, type_id);
    }
    return obj;
}
```

Data Flow:
1. VM requests allocation
2. GC attempts allocation from free list
3. If allocation fails, trigger collection
4. Retry allocation
5. Return object or NULL

Error Handling:
- Allocation failure after GC → Out of memory error
- Invalid size → Assertion failure
- Invalid type_id → Assertion failure

B. Root Set Scanning

Interface: `vm_scan_roots()`

```
// GC calls VM to scan roots
void vm_scan_roots(VM* vm, GCRootSet* roots) {
    // Scan VM stack
    for (double* slot = vm->stack; slot < vm->stack_top; slot++) {
        if (is_object_reference(*slot)) {
            GCObject* obj = (GCObject*)(*slot);
            gc_root_set_add(roots, obj);
        }
    }

    // Scan global variables
    for (size_t i = 0; i < vm->global_count; i++) {
        if (is_object_reference(vm->globals[i])) {
            GCObject* obj = (GCObject*)(vm->globals[i]);
            gc_root_set_add(roots, obj);
        }
    }

    // Scan call frames
    for (CallFrame* frame = vm->frames; frame < vm->frame_top; frame++) {
        // Scan frame locals
        for (size_t i = 0; i < frame->local_count; i++) {
            if (is_object_reference(frame->locals[i])) {
                GCObject* obj = (GCObject*)(frame->locals[i]);
                gc_root_set_add(roots, obj);
            }
        }
    }
}
```

Data Flow:
1. GC initiates collection
2. GC calls `vm_scan_roots()`
3. VM scans stack, globals, call frames
4. VM adds object references to root set
5. GC marks reachable objects

Challenges:
- Distinguishing object references from numeric values
- Handling tagged pointers
- Ensuring all roots are scanned

C. Write Barriers

Interface: `vm_write_barrier()`

```
// VM calls write barrier on object field update
void vm_write_barrier(VM* vm, GCObject* obj, GCObject* value) {
    if (obj->header.generation == GEN_OLD &&
        value->header.generation == GEN_YOUNG) {
        // Old object now references young object
        // Add to remembered set
        generational_gc_add_to_remembered_set(vm->gc, obj);
    }
}
```

Data Flow:
1. VM updates object field
2. VM calls write barrier
3. Write barrier checks generation
4. If old→young reference, add to remembered set
5. GC uses remembered set during young generation collection

Performance Considerations:
- Write barriers add overhead to every object field update
- Optimize for common case (no barrier needed)
- Use inline assembly for critical paths

D. GC Triggering

Interface: `vm_trigger_gc()`

```
// VM triggers GC collection
void vm_trigger_gc(VM* vm) {
    // Scan roots
    GCRootSet roots;
    gc_root_set_init(&roots);
    vm_scan_roots(vm, &roots);

    // Perform collection
    if (vm->gc_type == GC_GENERATIONAL) {
        generational_gc_collect(vm->gc, &roots);
    } else {
        mark_sweep_collect(vm->gc, &roots);
    }

    // Cleanup
    gc_root_set_free(&roots);
}
```

Trigger Conditions:
- Allocation failure
- Allocation threshold reached
- Explicit `vm_force_gc()` call
- Periodic collection (every N allocations)

1.3 Data Structures

VM-Side Structures

```
typedef struct {
    // ... existing VM fields ...

    // GC integration
    MarkSweepGC* gc;              // GC instance
    GCRootSet* gc_roots;          // Root set
    size_t allocation_count;      // Allocations since last GC
    size_t gc_threshold;          // Trigger GC after N allocations
    bool gc_enabled;              // GC on/off
} VM;
```

GC-Side Structures

```
typedef struct {
    // ... existing GC fields ...

    // VM integration
    void (*scan_roots_callback)(void* vm, GCRootSet* roots);
    void* vm_context;             // Opaque VM pointer
} MarkSweepGC;
```

1.4 Testing Strategy

Unit Tests:
- Test allocation through GC
- Test root set scanning
- Test write barriers
- Test GC triggering

Integration Tests:
- Allocate objects, verify collection
- Create garbage, verify reclamation
- Test old→young references
- Test allocation failure handling

Stress Tests:
- Allocate millions of objects
- Create complex object graphs
- Test under memory pressure

---

2. VM ↔ JIT Compiler Integration

2.1 Interface Overview

Purpose: Enable just-in-time compilation of hot bytecode paths to native code.

Status: ❌ Not Implemented (Planned for PR #107)

2.2 Integration Points

A. Hot Path Detection

Interface: `vm_check_hot_path()`

```
// VM checks if function is hot and should be compiled
bool vm_check_hot_path(VM* vm, uint32_t function_id) {
    FunctionProfile* profile = &vm->function_profiles[function_id];
    profile->execution_count++;

    if (profile->execution_count >= vm->jit_threshold &&
        !profile->is_compiled) {
        return true;  // Hot path detected
    }
    return false;
}
```

Data Flow:
1. VM executes function
2. VM increments execution counter
3. If counter exceeds threshold, mark as hot
4. Trigger JIT compilation
5. Cache compiled code

Threshold Tuning:
- Default: 1000 executions
- Configurable per function
- Adaptive thresholds based on function size

B. JIT Compilation Trigger

Interface: `vm_compile_function()`

```
// VM triggers JIT compilation
CompiledCode* vm_compile_function(VM* vm, uint32_t function_id) {
    // Get bytecode
    BCIChunk* chunk = vm_get_function_chunk(vm, function_id);

    // Compile to native code
    CompiledCode* code = NULL;
    JITStatus status = jit_compiler_compile_function(
        vm->jit_compiler,
        chunk,
        function_id,
        JIT_TIER_BASELINE,
        &code
    );

    if (status == JIT_STATUS_SUCCESS) {
        // Cache compiled code
        vm_cache_compiled_code(vm, function_id, code);
        return code;
    }

    return NULL;  // Compilation failed, fall back to interpreter
}
```

Data Flow:
1. VM detects hot path
2. VM calls JIT compiler
3. JIT translates bytecode to LLVM IR
4. LLVM optimizes IR
5. LLVM generates native code
6. VM caches native code
7. VM executes native code

Error Handling:
- Compilation failure → Fall back to interpreter
- Invalid bytecode → Assertion failure
- LLVM error → Log and fall back

C. Native Code Execution

Interface: `vm_execute_native()`

```
// VM executes compiled native code
int64_t vm_execute_native(VM* vm, CompiledCode* code, int64_t* args, size_t
arg_count) {
    // Set up execution context
    VMContext context = {
        .vm = vm,
        .stack = vm->stack,
        .globals = vm->globals,
        .gc = vm->gc
    };

    // Call native function
    int64_t result = code->native_code(&context, args, arg_count);

    // Update statistics
    code->execution_count++;

    return result;
}
```

Data Flow:
1. VM checks if function has compiled code
2. If yes, call native function
3. Pass VM context and arguments
4. Native code executes
5. Native code returns result
6. VM continues execution

Context Passing:
- VM context passed as first argument
- Allows native code to access VM state
- Enables calls back to VM (e.g., for allocation)

D. Deoptimization

Interface: `vm_deoptimize()`

```
// Native code calls back to VM to deoptimize
void vm_deoptimize(VMContext* context, uint32_t function_id, uint32_t
bytecode_offset) {
    VM* vm = context->vm;

    // Mark function for recompilation
    FunctionProfile* profile = &vm->function_profiles[function_id];
    profile->needs_recompilation = true;

    // Fall back to interpreter
    vm->ip = vm->chunk->code + bytecode_offset;
    vm_interpret_from_current_ip(vm);
}
```

Deoptimization Triggers:
- Type assumption violated
- Inline cache miss
- Guard failure
- Rare path taken

2.3 Data Structures

VM-Side Structures

```
typedef struct {
    uint32_t execution_count;
    bool is_compiled;
    bool needs_recompilation;
    CompiledCode* compiled_code;
    uint64_t total_time_ns;
} FunctionProfile;

typedef struct {
    // ... existing VM fields ...

    // JIT integration
    JITCompiler* jit_compiler;
    FunctionProfile* function_profiles;
    size_t function_count;
    uint32_t jit_threshold;
    bool jit_enabled;
} VM;
```

JIT-Side Structures

```
typedef struct {
    uint32_t function_id;
    CompiledFunction native_code;
    JITTier tier;
    uint64_t execution_count;
    bool needs_recompilation;
} CompiledCode;
```

2.4 Testing Strategy

Unit Tests:
- Test hot path detection

- Test compilation triggering
- Test native code execution
- Test deoptimization

Integration Tests:
- Compile and execute simple functions
- Test interpreter→native transition
- Test native→interpreter transition
- Test performance improvement

Performance Tests:
- Measure compilation overhead
- Measure execution speedup
- Compare interpreter vs JIT

---

3. Bytecode ↔ VM Integration

3.1 Interface Overview

Purpose: Enable VM to execute bytecode instructions.

Status: ✅ Partially Implemented (Basic opcodes working)

3.2 Integration Points

A. Bytecode Loading

Interface: `vm_load_chunk()`

```
// VM loads bytecode chunk
void vm_load_chunk(VM* vm, Chunk* chunk) {
    vm->chunk = chunk;
    vm->ip = chunk->code;
}
```

Data Flow:
1. Compiler generates bytecode chunk
2. VM loads chunk
3. VM sets instruction pointer to start
4. VM begins execution

B. Instruction Execution

Interface: `vm_execute_instruction()`

```
// VM executes single instruction
void vm_execute_instruction(VM* vm) {
    uint8_t opcode = *vm->ip++;

    switch (opcode) {
        case OP_CONSTANT: {
            uint8_t constant_index = *vm->ip++;
            double constant = vm->chunk->constants.data[constant_index];
            vm_stack_push(vm, constant);
            break;
        }
        case OP_ADD: {
            double b = vm_stack_pop(vm);
            double a = vm_stack_pop(vm);
            vm_stack_push(vm, a + b);
            break;
        }
        // ... more opcodes ...
    }
}
```

Execution Loop:

```
InterpretResult vm_interpret(VM* vm, Chunk* chunk) {
    vm_load_chunk(vm, chunk);

    while (true) {
        uint8_t opcode = *vm->ip;

        if (opcode == OP_RETURN) {
            return INTERPRET_OK;
        }

        vm_execute_instruction(vm);
    }
}
```

3.3 Bytecode Format

Instruction Format

```
┌──────────────┬──────────────┬──────────────┬──────────────┐
│   Opcode     │  Operand 1   │  Operand 2   │  Operand 3   │
│   (1 byte)   │  (0-4 bytes) │  (0-4 bytes) │  (0-4 bytes) │
└──────────────┴──────────────┴──────────────┴──────────────┘
```

Opcode Categories

Stack Operations:
- `OP_CONSTANT` - Push constant onto stack
- `OP_POP` - Pop value from stack
- `OP_DUP` - Duplicate top of stack

Arithmetic Operations:
- `OP_ADD`, `OP_SUBTRACT`, `OP_MULTIPLY`, `OP_DIVIDE`
- `OP_NEGATE` - Negate top of stack
- `OP_MODULO` - Modulo operation

Comparison Operations:
- `OP_EQUAL`, `OP_NOT_EQUAL`
- `OP_GREATER`, `OP_GREATER_EQUAL`
- `OP_LESS`, `OP_LESS_EQUAL`

Logical Operations:
- `OP_NOT` - Logical NOT
- `OP_AND` - Logical AND (short-circuit)
- `OP_OR` - Logical OR (short-circuit)

Control Flow:
- `OP_JUMP` - Unconditional jump
- `OP_JUMP_IF_FALSE` - Conditional jump
- `OP_LOOP` - Loop back jump
- `OP_CALL` - Function call
- `OP_RETURN` - Return from function

Variable Operations:
- `OP_GET_LOCAL`, `OP_SET_LOCAL` - Local variables
- `OP_GET_GLOBAL`, `OP_SET_GLOBAL` - Global variables
- `OP_GET_UPVALUE`, `OP_SET_UPVALUE` - Closure upvalues

3.4 Testing Strategy

Unit Tests:
- Test each opcode individually
- Test instruction decoding
- Test operand extraction

Integration Tests:
- Test instruction sequences
- Test control flow
- Test function calls

---

4. AST ↔ Bytecode Integration

4.1 Interface Overview

Purpose: Generate bytecode from abstract syntax tree.

Status: ⚠️ Partially Implemented (Basic expressions only)

4.2 Integration Points

A. Code Generation

Interface: `codegen_generate()`

```c
// Generate bytecode from AST
bool codegen_generate(CodeGenerator* codegen, AstNode* program) {
    // Traverse AST and emit bytecode
    codegen_visit_node(codegen, program);

    // Emit final return
    chunk_write_byte(codegen->compiling_chunk, OP_RETURN);

    return !codegen->had_error;
}
```

AST Traversal:

```c
void codegen_visit_node(CodeGenerator* codegen, AstNode* node) {
    switch (node->kind) {
        case AST_NODE_LITERAL:
            codegen_literal(codegen, node);
            break;
        case AST_NODE_BINARY_OP:
            codegen_binary_op(codegen, node);
            break;
        case AST_NODE_IF_STMT:
            codegen_if_statement(codegen, node);
            break;
        // ... more node types ...
    }
}
```

B. Expression Code Generation

Example: Binary Operation

```c
void codegen_binary_op(CodeGenerator* codegen, AstNode* node) {
    AstBinaryOp* binop = &node->data.binary_op;

    // Generate code for left operand
    codegen_visit_node(codegen, binop->left);

    // Generate code for right operand
    codegen_visit_node(codegen, binop->right);

    // Emit operation
    if (strcmp(binop->op, "+") == 0) {
        chunk_write_byte(codegen->compiling_chunk, OP_ADD);
    } else if (strcmp(binop->op, "-") == 0) {
        chunk_write_byte(codegen->compiling_chunk, OP_SUBTRACT);
    }
    // ... more operators ...
}
```

C. Statement Code Generation

Example: If Statement

```c
void codegen_if_statement(CodeGenerator* codegen, AstNode* node) {
    AstIfStmt* if_stmt = &node->data.if_stmt;

    // Generate condition
    codegen_visit_node(codegen, if_stmt->condition);

    // Emit conditional jump (to be patched)
    chunk_write_byte(codegen->compiling_chunk, OP_JUMP_IF_FALSE);
    size_t jump_offset = chunk_write_placeholder(codegen->compiling_chunk);

    // Generate then branch
    codegen_visit_node(codegen, if_stmt->then_branch);

    // Patch jump offset
    chunk_patch_jump(codegen->compiling_chunk, jump_offset);

    // Generate else branch (if exists)
    if (if_stmt->else_branch) {
        codegen_visit_node(codegen, if_stmt->else_branch);
    }
}
```

4.3 Testing Strategy

Unit Tests:
- Test code generation for each AST node type
- Test jump patching
- Test constant emission

Integration Tests:
- Generate bytecode for complete programs
- Execute generated bytecode
- Verify correct output

---

5. Graph ↔ VM Integration

5.1 Interface Overview

Purpose: Execute computational graphs in the VM.

Status: ❌ Not Implemented (Planned for PR #108)

5.2 Integration Points

A. Graph-to-Bytecode Lowering

Interface: `graph_lower_to_bytecode()`

```
// Lower graph to bytecode
Chunk* graph_lower_to_bytecode(BdiGraph* graph) {
    Chunk* chunk = chunk_create();

    // Topological sort for execution order
    NodeId* sorted_nodes = graph_topological_sort(graph);

    // Generate bytecode for each node
    for (size_t i = 0; i < graph->node_count; i++) {
        GraphNode* node = graph_get_node(graph, sorted_nodes[i]);
        graph_lower_node(chunk, node);
    }

    // Emit return
    chunk_write_byte(chunk, OP_RETURN);

    return chunk;
}
```

Node Lowering:

```
void graph_lower_node(Chunk* chunk, GraphNode* node) {
    switch (node->op) {
        case OP_CONST:
            // Emit constant load
            chunk_write_byte(chunk, OP_CONSTANT);
            chunk_write_byte(chunk, node->constant_index);
            break;

        case OP_ADD:
            // Inputs already on stack
            chunk_write_byte(chunk, OP_ADD);
            break;

        case OP_MATMUL:
            // Emit matrix multiply call
            chunk_write_byte(chunk, OP_CALL);
            chunk_write_byte(chunk, BUILTIN_MATMUL);
            break;

        // ... more operations ...
    }
}
```

B. Graph Execution

Interface: `vm_execute_graph()`

```
// Execute graph in VM
InterpretResult vm_execute_graph(VM* vm, BdiGraph* graph) {
    // Lower graph to bytecode
    Chunk* chunk = graph_lower_to_bytecode(graph);

    // Execute bytecode
    InterpretResult result = vm_interpret(vm, chunk);

    // Cleanup
    chunk_free(chunk);

    return result;
}
```

5.3 Testing Strategy

Unit Tests:
- Test graph-to-bytecode lowering
- Test node lowering for each operation

Integration Tests:
- Build graph, lower to bytecode, execute
- Verify correct results
- Test complex graphs

---

6. Graph ↔ Optimization Integration

6.1 Interface Overview

Purpose: Optimize computational graphs before execution.

Status: ❌ Not Implemented (Planned for PR #108)

6.2 Integration Points

A. Optimization Pass Framework

Interface: `graph_apply_optimization_pass()`

```
// Apply optimization pass to graph
bool graph_apply_optimization_pass(BdiGraph* graph, OptimizationPass* pass) {
    bool changed = false;

    // Iterate over nodes
    for (size_t i = 0; i < graph->node_count; i++) {
        GraphNode* node = &graph->nodes[i];

        // Apply pass to node
        if (pass->visit_node(graph, node)) {
            changed = true;
        }
    }

    return changed;
}
```

B. Constant Folding Pass

Example:

```
bool constant_folding_pass(BdiGraph* graph, GraphNode* node) {
    if (node->op == OP_ADD) {
        // Check if both inputs are constants
        GraphNode* left = graph_get_node(graph, node->inputs[0]);
        GraphNode* right = graph_get_node(graph, node->inputs[1]);

        if (left->op == OP_CONST && right->op == OP_CONST) {
            // Fold constants
            double result = left->constant_value + right->constant_value;

            // Replace node with constant
            node->op = OP_CONST;
            node->constant_value = result;
            node->input_count = 0;

            return true;  // Graph changed
        }
    }

    return false;  // No change
}
```

6.3 Testing Strategy

Unit Tests:
- Test each optimization pass individually
- Test pass framework

Integration Tests:
- Apply passes to graphs
- Verify correctness
- Verify performance improvement

---

7. Graph ↔ Device Backend Integration

7.1 Interface Overview

Purpose: Dispatch graph operations to appropriate devices (CPU/GPU/FPGA).

Status: ❌ Not Implemented (Planned for future)

7.2 Integration Points

A. Device Selection

Interface: `graph_select_device()`

```
// Select device for graph node
DeviceId graph_select_device(GraphNode* node) {
    // Use device hint if provided
    if (node->device_hint != DEVICE_AUTO) {
        return node->device_hint;
    }

    // Heuristic-based selection
    if (node->op == OP_MATMUL && node->size > 1024) {
        return DEVICE_GPU;  // Large matrix multiply → GPU
    } else if (node->flags & NODE_FLAG_SYNTHESIZE) {
        return DEVICE_FPGA;  // Synthesizable → FPGA
    } else {
        return DEVICE_CPU;  // Default → CPU
    }
}
```

B. Device Dispatch

Interface: `device_execute_node()`

```
// Execute node on device
void device_execute_node(Device* device, GraphNode* node) {
    switch (device->type) {
        case DEVICE_CPU:
            cpu_execute_node(device, node);
            break;
        case DEVICE_GPU:
            gpu_execute_node(device, node);
            break;
        case DEVICE_FPGA:
            fpga_execute_node(device, node);
            break;
    }
}
```

7.3 Testing Strategy

Unit Tests:
- Test device selection logic
- Test device dispatch

Integration Tests:
- Execute graphs on different devices
- Verify correctness
- Measure performance

---

8. Summary of Integration Status

Integration Point	Status	Priority	PR
VM ↔ GC	❌ Not Implemented	P0	#104
VM ↔ JIT	❌ Not Implemented	P1	#107
VM ↔ Bytecode	✅ Partial	P0	#105
AST ↔ Bytecode	⚠️ Partial	P0	#105
Source ↔ AST	✅ Working	-	-
Graph ↔ VM	❌ Not Implemented	P1	#108
Graph ↔ Optimization	❌ Not Implemented	P1	#108
Graph ↔ Device	❌ Not Implemented	P2	Future
Trainer ↔ Graph	❌ Not Implemented	P2	Future

---

9. Next Steps

1. **PR #104**: Implement VM ↔ GC integration
2. **PR #105**: Complete VM ↔ Bytecode integration
3. **PR #106**: Validate end-to-end pipeline
4. **PR #107**: Implement VM ↔ JIT integration
5. **PR #108**: Implement Graph ↔ VM integration

Conclusion

This document provides a comprehensive map of all integration points in the BDI system. Each integration point is documented with interfaces, data flows, data structures, and testing strategies. This will serve as a reference during Phase 8 implementation.